# LAB 08:

## Question#01:

## CODE:

```
#include <iostream>
using namespace std;

class Vehicle
{
private:
    string type;
    string make;
    string model;
    string color;
    int year;
    double milesDriven;

public:
    Vehicle(string type, string make, string model,
            string color, int year, double miles)
        : type(type), make(make), model(model),
          color(color), year(year), milesDriven(miles) {}

    string getType() const
    {
        return type;
    }
    string getMake() const
    {
        return make;
    }
    string getModel() const
    {
        return model;
    }
    string getColor() const
    {
        return color;
    }
    int getYear() const
    {
```

```
        return year;
    }
    double getMilesDriven() const
    {
        return milesDriven;
    }

    virtual void display() const
    {
        cout << "Type: " << type
             << " | Make: " << make
             << " | Model: " << model
             << " | Color: " << color
             << " | Year: " << year
             << " | Miles: " << milesDriven << "\n";
    }

    virtual ~Vehicle() {}
};

class GasVehicle : public Vehicle
{
private:
    double fuelTankSize;

public:
    GasVehicle(string type, string make, string model, string color,
               int year, double miles, double tankSize)
        : Vehicle(type, make, model, color, year, miles),
          fuelTankSize(tankSize) {}

    double getFuelTankSize() const
    {
        return fuelTankSize;
    }

    void display() const override
    {
        Vehicle::display();
        cout << "  Fuel Tank: " << fuelTankSize << "L\n";
    }
};

class ElectricVehicle : public Vehicle
{
```

Object Oriented Programming:

```
private:
    double energyStorage;

public:
    ElectricVehicle(string type, string make, string model, string color,
                    int year, double miles, double energy)
        : Vehicle(type, make, model, color, year, miles),
          energyStorage(energy) {}

    double getEnergyStorage() const { return energyStorage; }

    void display() const override
    {
        Vehicle::display();
        cout << "  Battery: " << energyStorage << " kWh\n";
    }
};

class HighPerformance : public GasVehicle
{
private:
    int horsePower;
    double topSpeed;

public:
    HighPerformance(string type, string make, string model, string color,
                    int year, double miles, double tankSize,
                    int hp, double topSpeed)
        : GasVehicle(type, make, model, color, year, miles, tankSize),
          horsePower(hp), topSpeed(topSpeed) {}

    int getHorsePower() const { return horsePower; }
    double getTopSpeed() const { return topSpeed; }

    void display() const override
    {
        GasVehicle::display();
        cout << "  HP: " << horsePower << " | Top Speed: " << topSpeed << "
km/h\n";
    }
};

class HeavyVehicle : public GasVehicle, public ElectricVehicle
{
private:
```

```
    double maxWeight;
    int numberOfWheels;
    double length;

public:
    HeavyVehicle(string type, string make, string model, string color,
                 int year, double miles,
                 double tankSize, double energy,
                 double maxWeight, int wheels, double length)
        : GasVehicle(type, make, model, color, year, miles, tankSize),
          ElectricVehicle(type, make, model, color, year, miles, energy),
          maxWeight(maxWeight), numberOfWheels(wheels), length(length) {}

    double getMaxWeight() const { return maxWeight; }
    int getNumberOfWheels() const { return numberOfWheels; }
    double getLength() const { return length; }

    void display() const override
    {
        GasVehicle::display();
        cout << "  Battery: " << ElectricVehicle::getEnergyStorage() << " kWh\n";
        cout << "  Max Weight: " << maxWeight << " tons"
             << " | Wheels: " << numberOfWheels
             << " | Length: " << length << " m\n";
    }
};

class SportsCar : public HighPerformance
{
public:
    string gearbox;
    string driveSystem;

public:
    SportsCar(string type, string make, string model, string color,
              int year, double miles, double tankSize,
              int hp, double topSpeed,
              string gearbox, string driveSystem)
        : HighPerformance(type, make, model, color, year, miles,
                          tankSize, hp, topSpeed),
          gearbox(gearbox), driveSystem(driveSystem) {}

    void display() const override
    {
        HighPerformance::display();
```

```
        cout << "  Gearbox: " << gearbox
             << " | Drive: " << driveSystem << "\n";
    }
};

class ConstructionTruck : public HeavyVehicle
{
public:
    string cargo; // cement, gravel, sand (public as per UML)

public:
    ConstructionTruck(string type, string make, string model, string color,
                      int year, double miles,
                      double tankSize, double energy,
                      double maxWeight, int wheels, double length,
                      string cargo)
        : HeavyVehicle(type, make, model, color, year, miles,
                       tankSize, energy, maxWeight, wheels, length),
          cargo(cargo) {}

    void display() const override
    {
        HeavyVehicle::display();
        cout << "  Cargo: " << cargo << "\n";
    }
};

class Bus : public HeavyVehicle
{
private:
    int numberOfSeats;

public:
    Bus(string type, string make, string model, string color,
        int year, double miles,
        double tankSize, double energy,
        double maxWeight, int wheels, double length,
        int seats)
        : HeavyVehicle(type, make, model, color, year, miles,
                       tankSize, energy, maxWeight, wheels, length),
          numberOfSeats(seats) {}

    int getNumberOfSeats() const { return numberOfSeats; }

    void display() const override
```

```
    {
        HeavyVehicle::display();
        cout << "  Seats: " << numberOfSeats << "\n";
    }
};

int main()
{

    cout << "========== Bus ==========\n";
    Bus bus(
        "Public Transit","Volvo","B8R",
        "Red", 2022, 45000.0, 200.0, 150.0, 18.0, 6, 12.5, 52);
    bus.display();

    cout << "\n========== Sports Car ==========\n";
    SportsCar car("Coupe", "Ferrari","F40", "Red", 2023, 5000.0, 90.0, 478,
324.0, "Manual",
         "Rear Wheel");
    car.display();

    cout << "========== Construction Truck ==========" << endl;
    ConstructionTruck truck( "Heavy Duty",
"Caterpillar",  "CT660",  "Yellow",  2021,  80000.0,
         400.0,  0.0, 30.0, 10,  9.0, "Gravel");
    truck.display();

    return 0;
}
```

# OUTPUT:

```
========== Bus ==========
Type: Public Transit | Make: Volvo | Model: B8R | Color: Red | Year: 2022 | Miles: 45000
  Fuel Tank: 200L
  Battery: 150 kWh
  Max Weight: 18 tons | Wheels: 6 | Length: 12.5 m
  Seats: 52

========== Sports Car ==========
Type: Coupe | Make: Ferrari | Model: F40 | Color: Red | Year: 2023 | Miles: 5000
  Fuel Tank: 90L
  HP: 478 | Top Speed: 324 km/h
  Gearbox: Manual | Drive: Rear Wheel
========== Construction Truck ==========
Type: Heavy Duty | Make: Caterpillar | Model: CT660 | Color: Yellow | Year: 2021 | Miles: 80000
  Fuel Tank: 400L
  Battery: 0 kWh
  Max Weight: 30 tons | Wheels: 10 | Length: 9 m
  Cargo: Gravel
PS D:\NED University\2nd Semester\Object Oriented Programming (OOPs)\Labs\Lab 08>
```

# Question#02:

# CODE:

```cpp
#include <iostream>
using namespace std;


class Character {
protected:
    string name;
    int    level;
    int    health;

public:
    Character(string name, int level, int health)
        : name(name), level(level), health(health) {}


    virtual void attack() const {
        cout << name << " attacks!\n";
    }

    virtual void display() const {
        cout << "Name: "   << name
             << " | Level: " << level
             << " | HP: "    << health << "\n";
    }

    string getName()  const { return name; }
    int    getLevel() const { return level; }
    int    getHealth()const { return health; }

    virtual ~Character() {}
};



class Warrior : public Character {
```

Object Oriented Programming:

```
protected:
    int    strength;
    string meleeWeapon;

public:
    Warrior(string name, int level, int health, int strength, string weapon)
        : Character(name, level, health),
          strength(strength), meleeWeapon(weapon) {}


    void slash() const {
        cout << name << " slashes with " << meleeWeapon
             << "! (Strength: " << strength << ")\n";
    }

    void attack() const override {
        slash();
    }

    void display() const override {
        Character::display();
        cout << "  [Warrior] Strength: " << strength
             << " | Weapon: " << meleeWeapon << "\n";
    }
};



class Mage : public Character {
protected:
    int    intelligence;
    string spellProficiency;

public:
    Mage(string name, int level, int health, int intelligence, string spell)
        : Character(name, level, health),
          intelligence(intelligence), spellProficiency(spell) {}


    void fireball() const {
        cout << name << " casts Fireball! ("
             << spellProficiency << " magic, INT: " << intelligence << ")\n";
    }

    void attack() const override {
```

```
        fireball();
    }

    void display() const override {
        Character::display();
        cout << "  [Mage] Intelligence: " << intelligence
             << " | Spell: " << spellProficiency << "\n";
    }
};



class Archer : public Character {
protected:
    int    dexterity;
    string rangedWeapon;

public:
    Archer(string name, int level, int health, int dexterity, string weapon)
        : Character(name, level, health),
          dexterity(dexterity), rangedWeapon(weapon) {}


    void rapidShot() const {
        cout << name << " fires Rapid Shot with " << rangedWeapon
             << "! (DEX: " << dexterity << ")\n";
    }

    void attack() const override {
        rapidShot();
    }

    void display() const override {
        Character::display();
        cout << "  [Archer] Dexterity: " << dexterity
             << " | Weapon: " << rangedWeapon << "\n";
    }
};



class NPC : public Character {
private:
    string movementPattern;
    string dialogue;
```

Object Oriented Programming:

```
public:
    NPC(string name, int level, int health,
        string movement, string dialogue)
        : Character(name, level, health),
          movementPattern(movement), dialogue(dialogue) {}


    void move() const {
        cout << name << " moves in pattern: " << movementPattern << "\n";
    }

    void speak() const {
        cout << name << " says: \"" << dialogue << "\"\n";
    }

    void attack() const override {
        cout << name << " (NPC) has no attack!\n";
    }

    void display() const override {
        Character::display();
        cout << "  [NPC] Movement: " << movementPattern
             << " | Dialogue: \"" << dialogue << "\"\n";
    }
};



class Mighty : public Warrior, public Mage {
private:
    string title;

public:
    Mighty(string name, int level, int health,
           int strength, string weapon,
           int intelligence, string spell,
           string title)
        : Warrior(name, level, health, strength, weapon),
          Mage(name, level, health, intelligence, spell),
          title(title) {}


    void attack() const override {
        cout << "=== " << title << " unleashes combined attack! ===\n";
```

Object Oriented Programming:

```
        slash();
        fireball();
    }

    void display() const override {

        cout << "Name: "    << Warrior::name
             << " | Level: " << Warrior::level
             << " | HP: "    << Warrior::health << "\n";
        cout << "  [Mighty - " << title << "]\n";
        cout << "  Strength: "     << strength
             << " | Weapon: "      << meleeWeapon << "\n";
        cout << "  Intelligence: " << intelligence
             << " | Spell: "       << spellProficiency << "\n";
    }


    string getName() const { return Warrior::name; }
};



int main() {

    cout << "========== Warrior ==========\n";
    Warrior w("Aragorn", 10, 250, 85, "Sword");
    w.display();
    w.attack();
    w.slash();

    cout << "\n========== Mage ==========\n";
    Mage m("Gandalf", 15, 180, 95, "Arcane");
    m.display();
    m.attack();
    m.fireball();

    cout << "\n========== Archer ==========\n";
    Archer a("Legolas", 12, 200, 90, "Longbow");
    a.display();
    a.attack();
    a.rapidShot();

    cout << "\n========== NPC ==========\n";
    NPC npc("Village Elder", 1, 50, "Patrol", "Beware the dark forest!");
    npc.display();
```

Made By: Faarid Amir Shaikh CT-25276 :: Section F

```
    npc.move();
    npc.speak();

    cout << "\n========== Mighty ==========\n";
    Mighty mighty("Zephyros", 20, 400,
                  90, "Runic Axe",
                  88, "Fire",
                  "The Arcane Warlord");
    mighty.display();
    mighty.attack();

    cout << "\n========== Polymorphism Demo ==========\n";

    Character* party[] = { &w, &m, &a };
    for (auto* c : party) {
        cout << c->getName() << ": ";
        c->attack();
    }

    return 0;
}
```

# OUTPUT:

Object Oriented Programming:

```
========== Warrior ==========
Name: Aragorn | Level: 10 | HP: 250
  [Warrior] Strength: 85 | Weapon: Sword
Aragorn slashes with Sword! (Strength: 85)
Aragorn slashes with Sword! (Strength: 85)

========== Mage ==========
Name: Gandalf | Level: 15 | HP: 180
  [Mage] Intelligence: 95 | Spell: Arcane
Gandalf casts Fireball! (Arcane magic, INT: 95)
Gandalf casts Fireball! (Arcane magic, INT: 95)

========== Archer ==========
Name: Legolas | Level: 12 | HP: 200
  [Archer] Dexterity: 90 | Weapon: Longbow
Legolas fires Rapid Shot with Longbow! (DEX: 90)
Legolas fires Rapid Shot with Longbow! (DEX: 90)

========== NPC ==========
Name: Village Elder | Level: 1 | HP: 50
  [NPC] Movement: Patrol | Dialogue: "Beware the dark forest!"
Village Elder moves in pattern: Patrol
Village Elder says: "Beware the dark forest!"

========== Mighty ==========
Name: Zephyros | Level: 20 | HP: 400
  [Mighty - The Arcane Warlord]
  Strength: 90 | Weapon: Runic Axe
  Intelligence: 88 | Spell: Fire
=== The Arcane Warlord unleashes combined attack! ===
Zephyros slashes with Runic Axe! (Strength: 90)
Zephyros casts Fireball! (Fire magic, INT: 88)

========== Polymorphism Demo ==========
Aragorn: Aragorn slashes with Sword! (Strength: 85)
Gandalf: Gandalf casts Fireball! (Arcane magic, INT: 95)
Legolas: Legolas fires Rapid Shot with Longbow! (DEX: 90)
PS D:\NED University\2nd Semester\Object Oriented Programming (OOPs)\Labs\Lab 08>
```

---

# Question#03:

# CODE:

```cpp
#include <iostream>
using namespace std;


class Account {
protected:
    double balance;

public:

    Account() {
        cout << "Enter initial balance: ";
        cin >> balance;
```

```
    }

    // Parameterized constructor
    Account(double balance) : balance(balance) {}


    virtual void deposit(double amount) {
        balance += amount;
        cout << "Deposited: " << amount << " | New Balance: " << balance << "\n";
    }

    virtual void withdraw(double amount) {
        if (amount > balance)
            cout << "Insufficient funds!\n";
        else {
            balance -= amount;
            cout << "Withdrawn: " << amount << " | New Balance: " << balance <<
"\n";
        }
    }

    void checkBalance() const {
        cout << "Balance: " << balance << "\n";
    }

    double getBalance() const { return balance; }

    virtual ~Account() {} // Virtual Destructor.
};



class InterestAccount : public Account {
protected:
    double interest;

public:

    InterestAccount() : Account() {
        cout << "Enter interest rate (default 0.30): ";
        cin >> interest;
    }

    // Parameterized constructor
    InterestAccount(double balance, double interest = 0.30)
```

```
        : Account(balance), interest(interest) {}


    void deposit(double amount) override {
        double withInterest = amount + (amount * interest);
        balance += withInterest;
        cout << "Deposited: " << amount
             << " + Interest(" << interest * 100 << "%): " << amount * interest
             << " | New Balance: " << balance << "\n";
    }
};



class ChargingAccount : public Account {
protected:
    double fee;

public:

    ChargingAccount() : Account() {
        cout << "Enter withdrawal fee (default 25): ";
        cin >> fee;
    }

    // Parameterized constructor
    ChargingAccount(double balance, double fee = 25.0)
        : Account(balance), fee(fee) {}


    void withdraw(double amount) override {
        double total = amount + fee;
        if (total > balance)
            cout << "Insufficient funds (amount + fee Rs." << fee << ")!\n";
        else {
            balance -= total;
            cout << "Withdrawn: " << amount
                 << " + Fee: " << fee
                 << " | New Balance: " << balance << "\n";
        }
    }
};
```

Object Oriented Programming:

```
class ACI : public InterestAccount, public ChargingAccount {
public:
    // Default constructor – takes user input
    ACI() {
        cout << "\n-- ACI Account Setup --\n";
        cout << "Enter balance: ";
        // balance is ambiguous
        cin >> InterestAccount::balance;
        ChargingAccount::balance = InterestAccount::balance;

        cout << "Enter interest rate (default 0.30): ";
        cin >> interest;

        cout << "Enter withdrawal fee (default 25): ";
        cin >> fee;
    }

    // Parameterized constructor
    ACI(double balance, double interest = 0.30, double fee = 25.0)
        : InterestAccount(balance, interest),
          ChargingAccount(balance, fee) {}


    void deposit(double amount) override {
        InterestAccount::deposit(amount);
        ChargingAccount::balance = InterestAccount::balance;
    }

    void withdraw(double amount) override {
        ChargingAccount::withdraw(amount);
        InterestAccount::balance = ChargingAccount::balance;
    }

    void checkBalance() const {
        cout << "Balance: " << InterestAccount::balance << "\n";
    }


    void transfer(double amount, Account& target) {
        if (amount > InterestAccount::balance) {
            cout << "Insufficient funds for transfer!\n";
            return;
        }
        InterestAccount::balance -= amount;
        ChargingAccount::balance  = InterestAccount::balance;
```

Object Oriented Programming:

```
        target.deposit(amount);
        cout << "Transferred " << amount << " to Account.\n";
    }


    void transfer(double amount, InterestAccount& target) {
        if (amount > InterestAccount::balance) {
            cout << "Insufficient funds for transfer!\n";
            return;
        }
        InterestAccount::balance -= amount;
        ChargingAccount::balance  = InterestAccount::balance;
        target.deposit(amount);     // deposit with interest applied
        cout << "Transferred " << amount << " to InterestAccount (with
interest).\n";
    }


    void transfer(double amount, ChargingAccount& target) {
        if (amount > InterestAccount::balance) {
            cout << "Insufficient funds for transfer!\n";
            return;
        }
        InterestAccount::balance -= amount;
        ChargingAccount::balance  = InterestAccount::balance;
        target.deposit(amount);
        cout << "Transferred " << amount << " to ChargingAccount.\n";
    }
};



int main() {

    cout << "======= Interest Account =======\n";
    InterestAccount ia(1000, 0.30);
    ia.checkBalance();
    ia.deposit(500);
    ia.checkBalance();
    ia.withdraw(200);
    ia.checkBalance();

    cout << "\n======= Charging Account =======\n";
    ChargingAccount ca(1000, 25);
    ca.checkBalance();
```

Made By: Faarid Amir Shaikh CT-25276 :: Section F

```
    ca.deposit(300);
    ca.checkBalance();
    ca.withdraw(200);
    ca.checkBalance();

    cout << "\n======= ACI Account =======\n";
    ACI aci(5000, 0.30, 25);
    aci.checkBalance();
    aci.deposit(1000);
    aci.checkBalance();
    aci.withdraw(500);
    aci.checkBalance();

    cout << "\n-- Transfer to generic Account --\n";
    Account plain(2000);
    plain.checkBalance();
    aci.transfer(300, plain);
    plain.checkBalance();

    cout << "\n-- Transfer to InterestAccount --\n";
    InterestAccount ia2(500, 0.30);
    ia2.checkBalance();
    aci.transfer(400, ia2);
    ia2.checkBalance();

    cout << "\n-- Transfer to ChargingAccount --\n";
    ChargingAccount ca2(800, 25);
    ca2.checkBalance();
    aci.transfer(200, ca2);
    ca2.checkBalance();

    return 0;
}
```

# OUTPUT:

```
======= Interest Account =======
Balance: 1000
Deposited: 500 + Interest(30%): 150 | New Balance: 1650
Balance: 1650
Withdrawn: 200 | New Balance: 1450
Balance: 1450

======= Charging Account =======
Balance: 1000
Deposited: 300 | New Balance: 1300
Balance: 1300
Withdrawn: 200 + Fee: 25 | New Balance: 1075
Balance: 1075

======= ACI Account =======
Balance: 5000
Deposited: 1000 + Interest(30%): 300 | New Balance: 6300
Balance: 6300
Withdrawn: 500 + Fee: 25 | New Balance: 5775
Balance: 5775

-- Transfer to generic Account --
Balance: 2000
Deposited: 300 | New Balance: 2300
Transferred 300 to Account.
Balance: 2300

-- Transfer to InterestAccount --
Balance: 500
Deposited: 400 + Interest(30%): 120 | New Balance: 1020
Transferred 400 to InterestAccount (with interest).
Balance: 1020

-- Transfer to ChargingAccount --
Balance: 800
Deposited: 200 | New Balance: 1000
Transferred 200 to ChargingAccount.
Balance: 1000
PS D:\NED University\2nd Semester\Object Oriented Programming (OOPs)\Labs\Lab 08>
```

# Question#04:

# CODE:

```cpp
#include <iostream>
using namespace std;


class Media {
protected:
    string title;
    int    year;
    bool   isBorrowed;

public:
    Media(string title, int year)
        : title(title), year(year), isBorrowed(false) {}
```

Object Oriented Programming:

```
    void borrow() {
        if (isBorrowed)
            cout << "\"" << title << "\" is already borrowed!\n";
        else {
            isBorrowed = true;
            cout << "\"" << title << "\" has been borrowed.\n";
        }
    }


    void returnMedia() {
        if (!isBorrowed)
            cout << "\"" << title << "\" was not borrowed!\n";
        else {
            isBorrowed = false;
            cout << "\"" << title << "\" has been returned.\n";
        }
    }

    virtual void display() const {
        cout << "Title: "  << title
             << " | Year: " << year
             << " | Status: " << (isBorrowed ? "Borrowed" : "Available") << "\n";
    }

    virtual ~Media() {}
};



class BookAttributes {
protected:
    string author;
    string genre;
    int    pages;

public:
    BookAttributes(string author, string genre, int pages)
        : author(author), genre(genre), pages(pages) {}
};



class MagazineAttributes {
```

Object Oriented Programming:

```
protected:
    int    issueNumber;
    string publisher;

public:
    MagazineAttributes(int issueNumber, string publisher)
        : issueNumber(issueNumber), publisher(publisher) {}
};



class DVDAttributes {
protected:
    string director;
    double duration;

public:
    DVDAttributes(string director, double duration)
        : director(director), duration(duration) {}
};



class Book : public Media, public BookAttributes {
public:
    Book(string title, int year, string author, string genre, int pages)
        : Media(title, year),
          BookAttributes(author, genre, pages) {}

    void display() const override {
        Media::display();
        cout << "  [Book] Author: " << author
             << " | Genre: "        << genre
             << " | Pages: "        << pages << "\n";
    }
};



class Magazine : public Media, public MagazineAttributes {
public:
    Magazine(string title, int year, int issueNumber, string publisher)
        : Media(title, year),
          MagazineAttributes(issueNumber, publisher) {}
```

Made By: Faarid Amir Shaikh CT-25276 :: Section F

Object Oriented Programming:

```
    void display() const override {
        Media::display();
        cout << "  [Magazine] Issue: " << issueNumber
             << " | Publisher: "       << publisher << "\n";
    }
};



class DVD : public Media, public DVDAttributes {
public:
    DVD(string title, int year, string director, double duration)
        : Media(title, year),
          DVDAttributes(director, duration) {}

    void display() const override {
        Media::display();
        cout << "  [DVD] Director: " << director
             << " | Duration: "      << duration << " mins\n";
    }
};



int main() {

    cout << "========== Library System ==========\n\n";

    Book b("The Hobbit", 1937, "J.R.R. Tolkien", "Fantasy", 310);
    Magazine mag("National Geographic", 2023, 215, "Nat Geo Partners");
    DVD dvd("Inception", 2010, "Christopher Nolan", 148);


    cout << "-- Library Catalog --\n";
    b.display();
    mag.display();
    dvd.display();


    cout << "\n-- Borrowing --\n";
    b.borrow();
    dvd.borrow();
    b.borrow();
```

Made By: Faarid Amir Shaikh CT-25276 :: Section F

```
    cout << "\n-- Updated Status --\n";
    b.display();
    dvd.display();


    cout << "\n-- Returning --\n";
    b.returnMedia();
    dvd.returnMedia();
    dvd.returnMedia();


    cout << "\n-- Final Status --\n";
    b.display();
    mag.display();
    dvd.display();


    cout << "\n-- Polymorphism Demo --\n";
    Media* library[] = { &b, &mag, &dvd };
    for (auto* item : library) {
        item->display();
        cout << "\n";
    }

    return 0;
}
```

# OUTPUT:

Object Oriented Programming:

```
========== Library System ==========

-- Library Catalog --
Title: The Hobbit | Year: 1937 | Status: Available
  [Book] Author: J.R.R. Tolkien | Genre: Fantasy | Pages: 310
Title: National Geographic | Year: 2023 | Status: Available
  [Magazine] Issue: 215 | Publisher: Nat Geo Partners
Title: Inception | Year: 2010 | Status: Available
  [DVD] Director: Christopher Nolan | Duration: 148 mins

-- Borrowing --
"The Hobbit" has been borrowed.
"Inception" has been borrowed.
"The Hobbit" is already borrowed!

-- Updated Status --
Title: The Hobbit | Year: 1937 | Status: Borrowed
  [Book] Author: J.R.R. Tolkien | Genre: Fantasy | Pages: 310
Title: Inception | Year: 2010 | Status: Borrowed
  [DVD] Director: Christopher Nolan | Duration: 148 mins

-- Returning --
"The Hobbit" has been returned.
"Inception" has been returned.
"Inception" was not borrowed!

-- Final Status --
Title: The Hobbit | Year: 1937 | Status: Available
  [Book] Author: J.R.R. Tolkien | Genre: Fantasy | Pages: 310
Title: National Geographic | Year: 2023 | Status: Available
  [Magazine] Issue: 215 | Publisher: Nat Geo Partners
Title: Inception | Year: 2010 | Status: Available
  [DVD] Director: Christopher Nolan | Duration: 148 mins
```

```
-- Polymorphism Demo --
Title: The Hobbit | Year: 1937 | Status: Available
  [Book] Author: J.R.R. Tolkien | Genre: Fantasy | Pages: 310

Title: National Geographic | Year: 2023 | Status: Available
  [Magazine] Issue: 215 | Publisher: Nat Geo Partners

Title: Inception | Year: 2010 | Status: Available
  [DVD] Director: Christopher Nolan | Duration: 148 mins

PS D:\NED University\2nd Semester\Object Oriented Programming (OOPs)\Labs\Lab 08>
```